

Pose ambiguity của target phẳng (Planar Pose Ambiguity)

PHỤ THUỘC A/02-homography-va-hình-hoc-phang (homography của một mặt phẳng và phép phân rã nó thành pose – chỗ hai nghiệm xuất hiện lần đầu); A/03-pnp-va-uoc-luong-tu-the (IPPE và sai số tái chiếu – chương này dùng liên tục cả hai).

TÓM TẮT

Chương này giải thích hiện tượng mà gần như ai làm marker cũng gặp và ít người gọi đúng tên: pose nhảy qua lại giữa hai giá trị dù camera và tag gần như đứng yên. Nguyên nhân không phải nhiễu, không phải bug, và không chữa được bằng bộ lọc: với một target *phẳng*, bài toán PnP có *đúng hai* nghiệm cục bộ, và cả hai đều giải thích ảnh gần như tốt ngang nhau. Hai nghiệm ấy là ảnh gương của nhau qua một mặt phẳng, chúng có cùng vị trí tâm gần đúng nhưng hướng nghiêng ngược dấu, và khoảng cách giữa chúng theo góc có thể lên tới hàng chục độ. Ambiguity tệ nhất đúng ở tư thế fronto-parallel — tức đúng tư thế của một robot tiếp cận dock thẳng trục. Nếu chỉ nhớ một điều: *đây là một tính chất của hình học phẳng, không phải một khiếm khuyết của thuật toán*, nên cách chữa triệt để là phá tính phẳng chứ không phải lọc mạnh hơn.

KIẾN THỨC CẦN CÓ TRƯỚC

- **target phẳng** — tập điểm điều khiển 3D cùng nằm trên một mặt phẳng. Một tag vuông đơn lẻ là target phẳng; bốn tag dán trên cùng tấm bảng cũng vẫn là target phẳng, dù có bao nhiêu điểm đi nữa. Đây là điểm quan trọng nhất của định nghĩa: *thêm điểm đồng phẳng không phá được tính phẳng*.
- **cực tiểu cục bộ (local minimum)** — một điểm mà hàm mục tiêu nhỏ hơn mọi điểm lân cận, nhưng không nhất thiết nhỏ nhất toàn cục. Bộ giải phi tuyến dừng ở cực tiểu cục bộ gần nhất với điểm khởi đầu, và nó không có cách nào biết còn cực tiểu nào khác ở xa.
- **sai số tái chiếu (reprojection error)** — tổng bình phương khoảng cách giữa góc đo được và góc dự đoán bởi một pose giả định, tính bằng pixel. Đây là hàm mục tiêu của PnP tinh chỉnh và là đại lượng duy nhất bộ giải “nhìn thấy”.
- **tỉ số ambiguity** — tỉ số giữa sai số tái chiếu của nghiệm tốt thứ hai và nghiệm tốt nhất, $\rho = e_2/e_1 \geq 1$. Gần 1 nghĩa là hai nghiệm giải thích ảnh tốt ngang nhau và không phân biệt được; lớn (chẳng hạn trên 3–5) nghĩa là nghiệm thứ nhất thắng rõ ràng.
- **IPPE — Infinitesimal Plane-based Pose Estimation**, thuật toán giải PnP cho target phẳng dạng đóng, và điểm đáng giá nhất của nó là nó trả về *cả hai* nghiệm kèm sai số tái chiếu của từng nghiệm [1].
- **đồng phẳng và không đồng phẳng** — tập điểm gọi là không đồng phẳng nếu không tồn tại một mặt phẳng chứa tất cả. Với tập điểm không đồng phẳng, bài toán PnP có nghiệm duy nhất — đây là kết quả trung tâm và là lối thoát của cả chương.

CÂU HỎI MỞ ĐẦU

- Q1. Camera và tag của bạn đứng yên hoàn toàn trên bàn. Pose trả về vẫn nhảy qua lại giữa hai giá trị cách nhau 20° vài lần mỗi giây. Nếu không phải nhiễu và không phải bug, thì đó là gì — và vì sao lấy trung bình hai giá trị ấy lại là việc tệ nhất có thể làm?
- Q2. Bạn dán thêm ba tag nữa lên cùng tấm bảng, tổng cộng bốn tag và mười sáu góc thay vì bốn. Bài toán ambiguity được cải thiện bao nhiêu, và vì sao câu trả lời là “gần như không”?
- Q3. Hai nghiệm của một target phẳng có quan hệ hình học rất cụ thể với nhau, không phải hai giá trị ngẫu nhiên. Quan hệ ấy là gì, và biết nó giúp gì cho việc chẩn đoán?
- Q4. Thuật toán nào cũng phải chọn một trong hai nghiệm. Nếu nó chọn bằng cách lấy nghiệm có sai số tái chiếu nhỏ hơn, thì trong trường hợp nào cách chọn ấy tệ hơn cả việc tung đồng xu?
- Q5. Có một cách khử ambiguity triệt để, không cần lọc, không cần thông tin thời gian, không cần cảm biến khác — và nó không nằm ở tầng phần mềm. Cách ấy là gì, và vì sao nó triệt để trong khi mọi cách khác chỉ là giảm thiểu?

1.1 Đặt vấn đề

Có một triệu chứng rất đặc trưng, và nếu bạn đã làm marker thì gần như chắc chắn đã thấy nó.

Bạn đặt camera trên một cái giá, dán một tag lên tường, không ai chạm vào gì cả. Bạn in pose ra màn hình. Vị trí tâm tag thì ổn định — nó dao động vài phần mười milimét, đúng như A/05 dự đoán. Nhưng góc thì không ổn định chút nào: nó nhảy giữa, chẳng hạn, $+12^\circ$ và -8° theo trục pitch, vài lần mỗi giây, một cách hoàn toàn ngẫu nhiên. Bạn thử tag khác, thử detector khác, thử camera khác. Triệu chứng không đổi.

Phản ứng đầu tiên gần như luôn là “nhiều quá lớn”, và nó dẫn tới một chuỗi việc làm không giúp gì: thêm ánh sáng, tăng độ phân giải, dùng subpixel tốt hơn, thêm bộ lọc thông thấp. Ba việc đầu làm nhiều giảm thật nhưng *tần suất nhảy vẫn nguyên* — chỉ có điều mỗi giá trị trong hai giá trị trở nên ổn định hơn. Việc thứ tư thì tệ nhất trong bốn: lọc thông thấp lấy trung bình hai giá trị và cho ra một giá trị thứ ba mà *không* phải nghiệm nào cả — một pose sai hoàn toàn nhưng trông rất mượt.

Nguyên nhân đã được biết từ lâu và nó không nằm ở nhiễu. Với một target phẳng, bài toán tìm pose từ ảnh có *hai* nghiệm cục bộ, và hiện tượng này được nhận diện ít nhất từ giữa những năm 1990 trong công trình của Oberkampf và cộng sự [2], rồi được phân tích hoàn chỉnh bởi Schweighofer và Pinz [3], và cuối cùng được đưa vào một thuật toán dạng đóng trả về cả hai nghiệm bởi Collins và Bartoli [1]. Ba nhóm ấy độc lập nhau và cách nhau ba thập niên, nên kết quả này thuộc loại đã được kiểm chứng kỹ — đây là một trong số ít khẳng định trong cây mà tôi tự tin ở mức cao.

Điều đáng nói là vì sao nó vẫn làm nhiều người bất ngờ. Lý do nằm ở chỗ hiện tượng này *không xuất hiện trong các tình huống mà người ta hay thử đầu tiên*. Khi bạn cầm tag trong tay và xoay qua xoay lại để thử, tag thường nghiêng khá nhiều, và ở góc nghiêng lớn thì một nghiệm thẳng rõ ràng nên bạn không thấy vấn đề. Vấn đề chỉ nổi lên khi tag ở tư thế gần vuông góc với trục quang — tức đúng khi bạn gắn nó lên một cái dock và cho robot tiếp cận thẳng trục. Nói cách khác: *hiện tượng này ẩn mình trong giai đoạn thử nghiệm và xuất hiện đúng lúc triển khai*.

Cách hiển nhiên hơn để chữa — và cách hầu như ai cũng thử — là chọn nghiệm có sai số tái chiếu nhỏ hơn. Cách này đúng về nguyên tắc và nó là cách mọi thư viện đang làm. Chỗ nó hỏng là khi hai sai số gần bằng nhau: lúc ấy việc “chọn cái nhỏ hơn” trở thành việc chọn theo nhiễu, và nó tệ hơn tung đồng xu ở một điểm — tung đồng xu ít nhất không giả vờ là một quyết định có cơ sở. Chương này dành mục 5 cho việc *đo* xem ta đang ở tình huống nào, và mục 6 cho việc thoát ra khỏi tình huống ấy.

TRỰC GIÁC

Cầm một tấm bìa hình chữ nhật và giơ trước mắt, hơi nghiêng đi một chút. Nhắm một mắt lại, và nhìn kỹ hình dạng bạn thấy: một hình thang, cạnh trên hơi ngắn hơn cạnh dưới.

Bây giờ nghiêng tấm bìa theo *chiều ngược lại*, cùng một góc. Bạn lại thấy một hình thang, và nếu góc đúng bằng nhau thì hình thang lần này là ảnh lật của hình thang lần trước. Với một mắt, và với một tấm bìa đối xứng như hình vuông của một cái tag, *hai hình thang ấy trông giống nhau*.

Đó là toàn bộ vấn đề. Mắt bạn — và camera — không có cách nào phân biệt “nghiêng ra xa ở trên” với “nghiêng ra xa ở dưới” chỉ từ hình dạng phẳng của bóng chiếu. Với người thật thì còn có bóng đổ, có tô bóng, có thị sai hai mắt; camera đơn nhìn một cái tag đen trắng thì không có gì trong số đó.

Bây giờ để ý một điều quan trọng: nếu bạn nghiêng tấm bìa *rất nhiều*, hai hình thang bắt đầu khác nhau rõ, vì cạnh gần và cạnh xa chênh nhau nhiều và phối cảnh không còn đối xứng nữa. Còn nếu tấm bìa gần như vuông góc với hướng nhìn, hai hình thang gần như trùng khít — và lúc đó bạn hoàn toàn không phân biệt được.

Cuối cùng, lối thoát: dựng một cái que vuông góc lên mặt tấm bìa. Bây giờ hai tư thế không còn giống nhau nữa, vì cái que chỉ về hai hướng khác hẳn. Cái que ấy chính là điểm *không đồng phẳng*, và đó là toàn bộ nội dung của mục 6.

1.2 Vì sao đúng hai nghiệm: hình học của phép lật

Trực giác trên đủ để tin, nhưng để dùng được thì cần biết hai nghiệm ấy quan hệ với nhau thế nào một cách chính xác.

Xét một target phẳng và một pose thật (R, t) đặt nó trong hệ camera. Gọi $\mathbf{n}$ là pháp tuyến của mặt phẳng tag trong hệ camera, và gọi $\mathbf{c}$ là hướng từ tâm camera tới tâm tag. Hai vectơ ấy xác định một mặt phẳng — gọi là *mặt phẳng lật*, chứa trục nhìn tới tâm tag và pháp tuyến của tag.

Nghiệm thứ hai, gọi là (R', t') , thu được bằng cách *lật* target quanh trục nằm trong mặt phẳng tag và vuông góc với mặt phẳng lật. Nói cách khác: pháp tuyến $\mathbf{n}$ được phản xạ gương qua đường nhìn $\mathbf{c}$. Nếu pháp tuyến thật nghiêng một góc α so với đường nhìn về một phía, thì pháp tuyến của nghiệm thứ hai nghiêng đúng góc α về phía bên kia [3, §III].

Từ mô tả ấy rút ra ngay ba hệ quả rất dùng được.

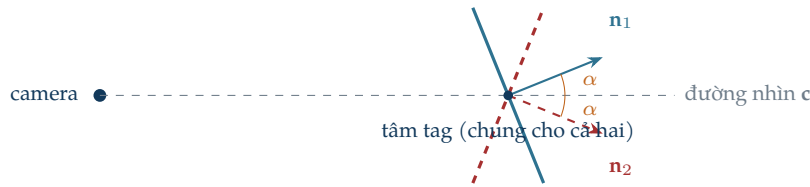
Hệ quả một: hai nghiệm cách nhau 2α về góc. Nếu tag nghiêng 10° so với fronto-parallel, hai nghiệm cách nhau 20° . Nếu tag nghiêng 30° , hai nghiệm cách nhau 60° — rất xa nhau, và cũng rất dễ phân biệt. Ngược lại, nếu tag gần fronto-parallel ($\alpha \rightarrow 0$) thì hai nghiệm *hội tụ về nhau* và đồng thời trở nên không phân biệt được. Đây là một quan hệ đáng chú ý vì nó đi theo hai chiều ngược nhau: ở góc nghiêng lớn, hai nghiệm xa nhau nhưng dễ phân biệt; ở góc nghiêng nhỏ, chúng gần nhau nhưng không phân biệt được.

Hệ quả hai: vị trí tâm gần như giống nhau ở cả hai nghiệm. Phép lật giữ nguyên (tới xấp xỉ bậc nhất) khoảng cách từ camera tới tâm tag, vì nó chỉ đổi hướng của mặt phẳng chứ không đổi vị trí tâm. Đây là lý do triệu chứng quan sát được là “vị trí ổn định, góc nhảy” chứ không phải cả sáu tham số cùng nhảy. Nó cũng là một manh mối chẩn đoán rất tốt: nếu bạn thấy góc nhảy mà vị trí đứng yên, thì gần như chắc chắn đó là ambiguity chứ không phải nhiễu chung.

Hệ quả ba, và đây là hệ quả thực dụng nhất: lấy trung bình hai nghiệm là vô nghĩa. Trung bình của hai xoay cách nhau 2α là một xoay ở chính giữa, tức xoay ứng với $\alpha = 0$, tức fronto-parallel. Nói cách khác, một bộ lọc thông thấp trên đầu ra pose sẽ luôn kéo bạn về phía kết luận “tag đang vuông

góc với camera”, bất kể tag thực sự nghiêng bao nhiêu. Đó không phải làm mượt; đó là tạo ra một thiên lệch có hệ thống hướng về một giá trị cụ thể và sai.

Hình 1.1 vẽ cấu hình này.



Hai nghiệm là **ảnh gương của nhau qua đường nhìn**. Chúng cách nhau 2α về góc và có *cùng* vị trí tâm tới xấp xỉ bậc nhất — nên triệu chứng là “góc nhảy, vị trí yên”. Khi $\alpha \rightarrow 0$ (fronto-parallel) hai nghiệm hội tụ *và* trở nên không phân biệt được: trung bình của chúng luôn là $\alpha = 0$, một giá trị sai có hệ thống.

Hình 1.1. Quan hệ hình học giữa hai nghiệm của một target phẳng, theo mô tả trong [3, §III]. Pháp tuyến của nghiệm thứ hai là ảnh phản xạ của pháp tuyến thật qua đường nhìn tới tâm tag. Hệ quả thực dụng quan trọng nhất nằm ở dòng cuối: mọi phép làm mượt trên đầu ra pose đều kéo kết quả về phía fronto-parallel, tức tạo ra một thiên lệch chứ không phải giảm nhiễu.

1.3 Vì sao thêm điểm đồng phẳng không cứu được

Đây là mục ngắn nhất của chương và cũng là mục hay bị bỏ qua nhất, nên nó đáng đứng riêng.

Phản xạ tự nhiên khi gặp ambiguity là thêm dữ liệu: dán thêm tag, dùng tag có nhiều điểm hơn, chuyển sang ChArUco board với hàng chục góc chessboard. Trực giác đằng sau là “nhiều ràng buộc hơn thì nghiệm xác định hơn”, và trực giác ấy đúng cho *nhiều* nhưng sai cho *ambiguity*.

Lý do nằm ở chỗ phép lật ở mục 2 là một phép biến đổi tác động lên *cả mặt phẳng*, không lên từng điểm riêng lẻ. Khi ta lật target, mọi điểm trên mặt phẳng đều được lật cùng nhau, và hình chiếu của toàn bộ tập điểm sau khi lật vẫn khớp với ảnh gần như tốt như trước. Thêm một điểm thứ năm, thứ mười, thứ trăm — miễn là nó nằm trên cùng mặt phẳng — thì nó cũng bị lật cùng, và nó cũng khớp. Số điểm không phá được một phép đối xứng của hình học.

Điều thêm điểm đồng phẳng *có* làm là giảm nhiễu của mỗi nghiệm. Với nhiều điểm hơn, cả hai nghiệm đều được xác định chính xác hơn, và sai số tái chiếu của cả hai đều nhỏ đi. Nhưng *tỉ* số giữa hai sai số ấy — đại lượng quyết định việc phân biệt — gần như không đổi. Bạn có hai nghiệm sắc nét hơn thay vì hai nghiệm nhoè, và bạn vẫn phải chọn một trong hai.

CẦN NHỚ

Số điểm không phá được ambiguity; chỉ có *tính không đồng phẳng* mới phá được.

Cụ thể: bốn tag dán trên cùng một tấm bảng phẳng vẫn là một target phẳng và vẫn có hai nghiệm. Hai tag dán trên hai mặt phẳng nghiêng với nhau thì không — và hai tag đã đủ. Đây là lý do vì sao lời khuyên ở 00-map không phải “dùng nhiều tag” mà là “dùng constellation *không đồng phẳng*”. Hai lời khuyên nghe gần giống nhau và chúng khác nhau về bản chất.

1.4 Khi nào ambiguity tệ nhất

Bốn yếu tố làm ambiguity nặng thêm, và biết chúng là biết cách tránh.

Góc nghiêng nhỏ (fronto-parallel). Đây là yếu tố mạnh nhất. Khi $\alpha \rightarrow 0$, hai nghiệm hội tụ và tỉ số sai số tái chiếu của chúng tiến về 1. Về mặt cơ chế, đây chính là hiện tượng mà A/05 mục 5 mô tả từ phía khác: độ nhạy của ảnh theo góc nghiêng tỉ lệ với $\sin \alpha$, nên khi α nhỏ thì hai giá trị góc rất khác nhau cho hai ảnh rất giống nhau. Hai chương nói cùng một sự thật: chương 5 nói “hiệp phương sai lớn”, chương này nói “có hai cực tiểu gần nhau”. Trên thực tế cả hai đều đúng và chúng là hai mặt của cùng một ma trận thông tin suy biến.

Tag nhỏ trên ảnh. Tag càng nhỏ thì chênh lệch sai số tái chiếu giữa hai nghiệm càng nhỏ so với nhiễu, vì chênh lệch ấy tỉ lệ với kích thước tag còn nhiễu thì không. Đây là lý do ambiguity nặng lên khi robot còn ở xa và nhẹ đi khi tới gần.

Trường nhìn hẹp, tức tiêu cự dài. Với ống kính tele, phép chiếu tiến gần tới phép chiếu trực giao (orthographic), và dưới phép chiếu trực giao thì hai nghiệm *trùng khít hoàn toàn* — ambiguity trở thành tuyệt đối chứ không chỉ khó phân biệt. Ống kính góc rộng cho nhiều biến dạng phối cảnh hơn, nên phân biệt tốt hơn. Đây là một đánh đổi đáng biết: tele cho f lớn nên sai số ngang nhỏ (A/05), nhưng nó làm ambiguity nặng hơn.

Tag nằm gần tâm ảnh. Một tag nằm ở rìa ảnh được nhìn dưới một góc chệch so với trục quang, và cái chệch ấy tự nó tạo ra một chút biến dạng phối cảnh phụ trội giúp phân biệt. Tag nằm chính giữa ảnh thì không có lợi thế đó. Hiệu ứng này yếu hơn ba yếu tố trên nhưng nó có thật, và nó gợi ý một mẹo bố trí: đừng đặt tag ở chính giữa trường nhìn nếu có thể tránh.

Bảng 1.1. Bốn yếu tố làm ambiguity nặng thêm, cơ chế, và cách giảm nhẹ ở tầng thiết kế.

Yếu tố	Cơ chế	Cách giảm nhẹ
Góc nghiêng nhỏ	Độ nhạy $\propto \sin \alpha$; hai nghiệm hội tụ khi $\alpha \rightarrow 0$	Nghiêng tag trong constellation; hoặc cho đường tiếp cận chệch
Tag nhỏ trên ảnh	Chênh lệch reprojection tỉ lệ với w , nhiễu thì không	Tag lớn hơn ở cự ly xa; nested multi-scale (B/09)
Trường nhìn hẹp (tele)	Tiến tới chiếu trực giao, nơi hai nghiệm trùng khít	Ống kính rộng hơn; chấp nhận đánh đổi với f nhỏ hơn
Tag ở tâm ảnh	Không có biến dạng phối cảnh phụ trội từ vị trí chệch	Bố trí tag lệch khỏi tâm trường nhìn

1.5 Đo ambiguity: tỉ số sai số tái chiếu

Nếu ambiguity không chứa được bằng phần mềm thì ít nhất nó phải *đo* được bằng phần mềm, và may mắn là có một đại lượng rất tự nhiên cho việc đó.

IPPE trả về hai nghiệm kèm sai số tái chiếu của từng nghiệm [1]. Định nghĩa tỉ số ambiguity:

$$\rho = \frac{e_2}{e_1}, \quad e_1 \leq e_2,$$

với e_1 là sai số tái chiếu của nghiệm tốt nhất và e_2 của nghiệm còn lại. Theo định nghĩa $\rho \geq 1$.

Đọc ρ như sau. Khi ρ lớn — chẳng hạn $\rho > 3$ — nghiệm thứ nhất giải thích ảnh tốt hơn hẳn và ta tin nó. Khi $\rho \approx 1$, hai nghiệm giải thích ảnh tốt ngang nhau, và việc chọn nghiệm nhỏ hơn là chọn theo nhiễu; kết quả sẽ nhảy từ khung này sang khung khác đúng như triệu chứng ở mục 1.

Ngưỡng đặt ở đâu thì đúng? Câu trả lời trung thực là *tôi không có một con số có cơ sở lý thuyết*, và tôi ghi rõ điều đó ở đây thay vì đưa ra một con số trông thuyết phục. Cái có cơ sở là cách *dẫn ra* ngưỡng cho hệ của bạn: chạy Monte Carlo (C/16), với mỗi tư thế ghi lại cả ρ lẫn việc nghiệm được chọn có đúng là nghiệm thật không, rồi vẽ tỉ lệ chọn sai theo ρ . Ngưỡng của bạn là giá trị

ρ mà dưới nó tỉ lệ chọn sai vượt mức bạn chấp nhận được. Ngưỡng ấy phụ thuộc vào σ , vào w , vào số điểm — nên nó phải là ngưỡng của hệ bạn chứ không phải một hằng số chép từ đâu đó.

A. Ví dụ số: chênh lệch reprojection lớn cỡ nào

Để thấy các con số ở bậc nào, làm một ước lượng thô. Giả sử tag 120 px trên ảnh (ví dụ chuẩn của cây, A/05 mục 4), nghiêng $\alpha = 5^\circ$, và $\sigma = 0,2$ px.

Nghiệm sai lệch khỏi nghiệm thật một góc $2\alpha = 10^\circ$. Sai lệch góc ấy gây ra sai lệch trong hình chiếu ở bậc: các điểm ở mép tag lệch khỏi vị trí đúng chừng $(w/2)(1 - \cos 2\alpha)$ theo chiều vuông góc với trục lật, tức $60 \times (1 - \cos 10^\circ) = 60 \times 0,0152 = 0,91$ px. Nhưng phần lớn sai lệch ấy được hấp thụ bằng cách điều chỉnh các tham số pose khác (khoảng cách, vị trí ngang), nên phần dư thực sự còn lại nhỏ hơn nhiều — đây chính là lý do nghiệm thứ hai tồn tại như một cực tiểu chứ không bị loại ngay.

Ý nghĩa của phép ước lượng này không nằm ở con số cụ thể mà ở bậc: *chênh lệch reprojection giữa hai nghiệm ở góc nghiêng nhỏ nằm cùng bậc với nhiễu định vị góc*. Khi hai đại lượng cùng bậc thì phân biệt chúng là việc bất khả thi theo nghĩa thống kê, không phải theo nghĩa “chưa đủ thông minh”. (Tôi suy ra ước lượng bậc này từ hình học; nó chưa được kiểm bằng mô phỏng — đã ghi vào 99-chua-biet.md.)

CẠM BẮY

Ba sai lầm phổ biến khi xử lý ambiguity, xếp theo mức độ tai hại tăng dần.

Nhệ: chỉ lấy nghiệm thứ nhất mà không nhìn ρ . Hệ thống chạy đúng phần lớn thời gian và sai một cách bất ngờ ở đúng những tư thế quan trọng nhất.

Nặng: lọc thông thấp trên góc để làm mượt. Như mục 2 đã chỉ ra, trung bình hai nghiệm luôn là fronto-parallel, nên bộ lọc biến một đại lượng *nhảy nhưng đôi khi đúng* thành một đại lượng *mượt và sai có hệ thống*. Tệ hơn nữa, nó xoá mất triệu chứng nên bạn không còn biết mình đang có vấn đề.

Rất nặng: chọn nghiệm theo tiêu chí “gần với pose khung trước”. Nghe rất hợp lý và nó là một kỹ thuật hợp lệ (B/10 mục về nhất quán thời gian), nhưng nếu dùng một mình thì nó có một chế độ hỏng chết người: một khi đã chọn nhầm nghiệm một lần, tiêu chí “gần khung trước” sẽ giữ bạn ở nghiệm sai đó mãi mãi. Bạn có một hệ thống rất ổn định quanh một giá trị sai, và không có tín hiệu nào báo động.

1.6 Lối thoát triệt để: phá tính đồng phẳng

Mọi thứ ở trên là giảm thiểu. Mục này là cách khử.

Kết quả then chốt: *với một tập điểm điều khiển không đồng phẳng, bài toán PnP có nghiệm duy nhất* (bỏ qua các trường hợp suy biến rất đặc biệt và với đủ điểm). Lý do đã nằm sẵn trong hình học của mục 2: phép lật là một phép đối xứng của mặt phẳng, và nó chỉ bảo toàn hình chiếu vì mọi điểm cùng nằm trên mặt phẳng ấy. Một điểm nằm ngoài mặt phẳng bị phép lật đưa tới một vị trí khác hẳn, hình chiếu của nó lệch đi rõ rệt, và sai số tái chiếu của nghiệm thứ hai nhảy vọt. Nghiệm thứ hai không còn là một cực tiểu nữa — nó bị loại bởi chính dữ liệu.

Điều đáng chú ý là *không cần nhiều* tính không đồng phẳng. Về mặt cấu trúc, chỉ cần một điểm nằm ngoài mặt phẳng là đủ để phá đối xứng. Về mặt thực hành, ta cần độ lệch khỏi mặt phẳng đủ lớn để chênh lệch reprojection vượt hẳn nhiễu — và đây là một câu hỏi định lượng mà B/09 trả lời bằng thiết kế cụ thể, còn C/16 trả lời bằng mô phỏng.

Ba cách hiện thực hoá, xếp theo độ dễ thi công.

Cách một, nghiêng các tag. Giữ tất cả tag trên cùng một tấm bảng nhưng gắn mỗi tag lên một nêm nghiêng $15\text{--}30^\circ$ theo các hướng khác nhau. Rẻ, và nó vừa phá đồng phẳng vừa giải quyết vấn đề $\sin \alpha$ của A/05: mỗi tag bây giờ có góc nghiêng riêng khác không so với trục quang, nên chính nó cũng có độ nhạy góc tốt hơn.

Cách hai, đặt tag ở các độ sâu khác nhau. Một số tag trên mặt phẳng dock, một số trên các gờ nhô ra 5–10 cm. Thi công dễ nếu dock vốn đã có cấu trúc ba chiều, và nó cho baseline theo chiều sâu — một hướng mà cách một không cho.

Cách ba, target ba chiều có chủ ý. Một cấu trúc dạng góc (hai hoặc ba mặt phẳng vuông góc nhau, mỗi mặt mang tag). Đây là cách mạnh nhất về mặt hình học nhưng nó chiếm không gian và dễ bị che khuất một phần khi robot tới gần.

CẦN NHỚ

Ambiguity là tính chất của *hình học phẳng*, không phải của thuật toán. Ba hệ quả:
 Một, không thuật toán nào khử được nó khi target còn phẳng — chỉ chọn được nghiệm, và việc chọn có xác suất sai khác không.
 Hai, cách khử duy nhất triệt để là phá tính đồng phẳng, và việc đó xảy ra ở xương cơ khí chứ không trong trình biên dịch.
 Ba, chi phí của việc khử rất thấp so với lợi ích: một nêm nghiêng 20° dưới mỗi tag là một chi tiết gia công tầm thường, và nó đồng thời giải quyết cả bài toán ambiguity lẫn bài toán độ nhạy góc.

1.7 Giới hạn và chế độ hỏng

Ba giới hạn của cách hiểu trình bày trong chương này.

“Hai nghiệm” là kết quả cho target phẳng dưới mô hình pinhole không nhiễu. Trong thực tế có nhiễu, và khi nhiễu lớn thì bề mặt hàm mục tiêu có thể có nhiều hơn hai cực tiểu cục bộ nông. Điều này hiếm và nó không đối bức tranh lớn, nhưng nó có nghĩa là một bộ giải lặp khởi đầu từ một điểm tôi có thể dừng ở chỗ không phải nghiệm nào trong hai nghiệm chính. Đây là lý do nên dùng một lời giải dạng đóng (IPPE, P3P) để khởi tạo thay vì khởi tạo tùy tiện — A/03 bàn kỹ.

Phép lật ở mục 2 là mô tả xấp xỉ. Quan hệ “ảnh gương qua đường nhìn” đúng chính xác trong giới hạn chiếu trục giao và là xấp xỉ tốt dưới chiếu phối cảnh với tag nhỏ so với khoảng cách. Với tag lớn chiếm phần lớn trường nhìn, quan hệ giữa hai nghiệm phức tạp hơn, và hai nghiệm cũng khác nhau ở cả vị trí tâm chứ không chỉ ở hướng. Trong bối cảnh docking thì xấp xỉ này ổn, vì tag hiếm khi chiếm quá một phần tư trường nhìn.

Không đồng phẳng khử ambiguity nhưng không khử được kém điều kiện. Một constellation “không đồng phẳng” với độ lệch khỏi mặt phẳng chỉ 2 mm về mặt kỹ thuật là không đồng phẳng, nhưng chênh lệch reprojection mà 2 mm ấy tạo ra có thể nhỏ hơn nhiễu, và khi đó bạn vẫn có hai cực tiểu gần như bằng nhau trong thực tế. Ranh giới giữa “đủ không đồng phẳng” và “không đủ” là một câu hỏi định lượng phụ thuộc σ , w và Z ; tôi không có công thức cho nó và tôi khuyên đo bằng mô phỏng — đây là một mục trong 99-chua-biet.md.

1.8 Thực hành

Bốn việc cụ thể, theo thứ tự nên làm.

Một, luôn tính cả hai nghiệm và luôn ghi ρ vào log. Dùng solvePnPGeneric với SOLVEPNP_IPPE_SQUARE cho tag vuông, hoặc đọc trực tiếp apriltag_pose.c nếu dùng thư viện gốc — nó đã tính sẵn cả hai. Chi phí gần như bằng không và giá trị chẩn đoán rất cao.

Hai, từ chối cập nhật khi ρ dưới ngưỡng thay vì trả về một nghiệm không đáng tin. Một hệ thống nói “tôi không biết” hữu ích hơn nhiều so với một hệ thống đoán và không báo. B/10 bàn chiến lược cụ thể.

Ba, đừng lọc thông thấp trên góc cho tới khi đã khử ambiguity. Nếu buộc phải làm mượt, hãy làm mượt sau khi đã chọn nghiệm một cách tin cậy, và hãy làm mượt trên đại lượng có ý nghĩa (B/11 về trung bình trên đa tạp), không phải trên góc Euler.

Bốn, và quan trọng nhất, hãy đưa yêu cầu “không đồng phẳng” vào bản vẽ cơ khí ngay từ đầu. Thêm một nêm nghiêng vào bản vẽ trước khi gia công tốn vài phút; thêm nó sau khi đã gia công một trăm cái dock thì tốn một trăm lần chi phí ấy.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vì sao target phẳng có đúng hai nghiệm	Vì phép lật target quanh trục trong mặt phẳng (phản xạ pháp tuyến qua đường nhìn) bảo toàn hình chiếu tới xấp xỉ bậc nhất. Đã được ba nhóm độc lập xác nhận [1]–[3].
••• Quan hệ giữa hai nghiệm	Ảnh gương qua đường nhìn; cách nhau 2α về góc với α là góc nghiêng thật; cùng vị trí tâm tới xấp xỉ bậc nhất.
••• Triệu chứng đặc trưng	Góc nhảy giữa hai giá trị, vị trí đứng yên. Nếu thấy cả sáu tham số cùng nhảy thì đó là nhiễu chứ không phải ambiguity.
••• Vì sao lọc thông thấp là sai	Trung bình hai nghiệm cách nhau 2α luôn ra $\alpha = 0$, tức fronto-parallel. Bộ lọc biến một đại lượng nhảy-nhưng-đôi-khi-đúng thành mượt-và-sai-có-hệ-thống, đồng thời xoá mất triệu chứng.
••• Thêm điểm đồng phẳng cứu được không?	Không. Phép lật tác động lên cả mặt phẳng nên mọi điểm đồng phẳng đều bị lật cùng và đều vẫn khớp. Nhiều điểm làm mỗi nghiệm sắc nét hơn nhưng ρ gần như không đổi.
••• Điều kiện khử triệt để	Tập điểm không đồng phẳng $\Rightarrow$ nghiệm duy nhất. Một điểm ngoài mặt phẳng là đủ về cấu trúc; thực hành cần độ lệch đủ lớn để chênh lệch reprojection vượt nhiễu.
••• Bốn yếu tố làm ambiguity nặng	Góc nghiêng nhỏ (mạnh nhất); tag nhỏ trên ảnh; trường nhìn hẹp (tele, tiến tới chiều trục giao); tag nằm ở tâm ảnh.
••• Tỉ số ambiguity	$\rho = e_2/e_1 \geq 1$, từ solvePnPGeneric hoặc IPPE. ρ gần 1 nghĩa là không phân biệt được và việc chọn nghiệm nhỏ hơn là chọn theo nhiễu.
•• Ngưỡng ρ nên đặt ở đâu	Không có hằng số phổ quát. Phải dẫn ra cho hệ của mình bằng Monte Carlo: vẽ tỉ lệ chọn sai theo ρ , lấy ngưỡng ở mức sai chấp nhận được. Phụ thuộc σ , w , số điểm.
•• Vì sao ống kính tele làm nặng thêm	Tele tiến gần tới chiều trục giao, mà dưới chiều trục giao hai nghiệm trùng khít hoàn toàn. Đánh đổi đáng biết: tele cho f lớn nên δx nhỏ, nhưng ambiguity nặng hơn.
•• Chế độ hồng của “chọn theo khung trước”	Một khi chọn nhầm, tiêu chí ấy giữ bạn ở nghiệm sai vĩnh viễn. Hệ thống rất ổn định quanh một giá trị sai và không có tín hiệu báo động. Phải kết hợp với ρ .
•• Quan hệ với chương 5	Cùng một sự thật ở hai mức: chương 5 nói hiệp phương sai theo hướng nghiêng lớn, chương này nói có hai cực tiểu gần nhau. Cả hai đều là hệ quả của $\sin \alpha \rightarrow 0$.
• Ba cách phá đồng phẳng	Nêm nghiêng dưới mỗi tag (rẻ nhất, giải quyết luôn bài toán $\sin \alpha$); tag ở các độ sâu khác nhau (cho baseline theo chiều sâu); target dạng góc nhiều mặt phẳng (mạnh nhất, dễ bị che khuất).

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
apriltag <code>apriltag_pose.c</code>	Cài IPPE, tính cả hai nghiệm và sai số tái chiếu của từng nghiệm; hàm <code>estimate_tag_pose</code> trả về tỉ số	Chỗ tốt nhất để đọc chương này thành code; vài trăm dòng
OpenCV <code>SOLVEPNP_IPPE_SQUARE</code> STag [4]	Cài [1] cho trường hợp riêng tag vuông; dùng với <code>solvePnPGeneric</code> để lấy cả hai nghiệm Thêm một đường tròn ngoại tiếp vào thiết kế tag để ổn định pose — tức thêm thông tin hình học thay vì đổi thuật toán	API duy nhất cho bạn ρ trực tiếp Cùng triết lý với chương này, giải quyết ở tầng thiết kế marker
ChArUco board tagslam [5]	Nhiều góc chessboard nhưng <i>vẫn đồng phẳng</i> — cải thiện σ , không cải thiện ρ Map nhiều tag ở các mặt phẳng khác nhau trong môi trường, rồi giải hợp nhất	Minh hoạ trực tiếp cho mục 3; đừng nhầm hai loại cải thiện Không đồng phẳng ở quy mô phòng; mức tin C

MINI PROJECT · Nhìn tận mắt hai nghiệm, và đo ngưỡng ρ cho hệ của mình · 1–2 ngày

Mục tiêu	chuyển ambiguity từ một khái niệm thành một đại lượng bạn đo được, và có một ngưỡng ρ có cơ sở thay vì một con số chép từ internet.
Dữ liệu	mô phỏng hoàn toàn, nối tiếp script của A/05. Không cần phần cứng.
Các bước	sinh một tag vuông ở góc nghiêng α đã biết; chiếu, thêm nhiễu σ ; gọi <code>solvePnPGeneric</code> với <code>SOLVEPNP_IPPE_SQUARE</code> và lấy <i>cả hai</i> nghiệm cùng sai số tái chiếu; ghi lại ρ , và ghi lại nghiệm nào gần chân trị hơn; lặp 2000 lần cho mỗi α , quét α từ 0° tới 45° ; vẽ hai đường: ρ trung vị theo α , và tỉ lệ chọn sai theo α ; cuối cùng vẽ tỉ lệ chọn sai theo ρ (gộp mọi α) — đó chính là đường cong cho bạn ngưỡng.
Xong khi	bạn có một con số ngưỡng ρ^* cho hệ của mình kèm câu phát biểu dạng “với $\rho > \rho^*$, tỉ lệ chọn nhầm nghiệm dưới 1%”. Kiểm tra chéo bắt buộc: lặp lại toàn bộ với constellation không đồng phẳng (thêm một tag nghiêng 20°) và xác nhận rằng ρ bây giờ lớn ở <i>mọi</i> α — nếu không thì độ lệch khỏi mặt phẳng của bạn chưa đủ, và bạn vừa đo được nó cần bao nhiêu.
Nối với	B/10 dùng trực tiếp ngưỡng này; B/09 dùng kết quả kiểm chéo để chọn góc nghiêng cho constellation.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Đó là pose ambiguity: bài toán PnP với target phẳng có đúng hai nghiệm cục bộ, cả hai giải thích ảnh gần như tốt ngang nhau, và nhiều quyết định nghiệm nào có sai số tái chiếu nhỏ hơn ở từng ảnh. Lấy trung bình là việc tệ nhất vì hai nghiệm là ảnh gương của nhau qua đường nhìn, nên trung bình của chúng luôn rơi đúng vào tư thế fronto-parallel — một giá trị không phải nghiệm nào cả và sai một cách có hệ thống. Tệ hơn nữa, nó xoá mất triệu chứng nhảy, nên bạn mất luôn dấu hiệu duy nhất cho biết mình đang có vấn đề.
- A2. Gần như không, và đây là điểm hay bị hiểu nhầm nhất. Phép lật sinh ra nghiệm thứ hai tác động lên *cả mặt phẳng*, nên mọi điểm nằm trên mặt phẳng ấy đều bị lật cùng nhau và đều vẫn khớp với ảnh. Mười sáu góc đồng phẳng cho hai nghiệm sắc nét hơn bốn góc đồng phẳng, nhưng *tỉ số* ρ giữa hai sai số tái chiếu — đại lượng quyết định việc phân biệt — gần như không đổi. Bạn cần tính không đồng phẳng, không phải số lượng: hai tag trên hai mặt phẳng nghiêng nhau thắng bốn tag trên một mặt phẳng.
- A3. Nghiệm thứ hai là ảnh gương của nghiệm thật qua đường nhìn tới tâm tag: nếu pháp tuyến thật nghiêng α về một phía thì pháp tuyến nghiệm hai nghiêng α về phía kia, nên hai nghiệm cách nhau 2α về góc và có cùng vị trí tâm tới xấp xỉ bậc nhất [3, §III]. Biết quan hệ này giúp ba việc: nhận diện triệu chứng (góc nhảy, vị trí yên — khác hẳn với nhiễu chung làm cả sáu

tham số nhảy); dự đoán biên độ nhảy (bằng hai lần góc nghiêng, nên biên độ nhảy nhỏ lại là dấu hiệu *xấu* vì nó nghĩa là bạn đang ở gần fronto-parallel); và hiểu vì sao trung bình luôn ra fronto-parallel.

- A4. Khi $\rho \approx 1$, tức khi hai sai số tái chiếu gần bằng nhau. Lúc ấy cái quyết định nghiệm nào nhỏ hơn là nhiều chứ không phải hình học, nên chọn theo ϵ nhỏ hơn về bản chất là tung đồng xu. Nó tệ hơn tung đồng xu ở một điểm quan trọng: tung đồng xu không giả vờ là một quyết định có cơ sở, còn “chọn nghiệm có residual nhỏ hơn” thì nghe rất khoa học và không ai nghĩ tới việc kiểm lại. Cách sửa không phải đổi tiêu chí chọn mà là *đo ρ và từ chối trả lời khi nó quá thấp*.
- A5. Phá tính đồng phẳng của tập điểm — nghiêng các tag, đặt chúng ở các độ sâu khác nhau, hoặc dùng target dạng góc. Nó triệt để vì nó phá chính phép đối xứng sinh ra nghiệm thứ hai: điểm nằm ngoài mặt phẳng bị phép lật đưa tới vị trí khác hẳn, hình chiếu của nó lệch rõ rệt, nên nghiệm thứ hai không còn là cực tiểu nữa mà bị dữ liệu loại bỏ. Mọi cách khác — ngưỡng ρ , nhất quán thời gian, fuse IMU — chỉ giúp *chọn đúng* trong hai nghiệm vẫn đang tồn tại, nên chúng luôn có một xác suất sai khác không. Và điểm đáng nói về kinh tế: cách triệt để này được thi công ở xương cơ khí với chi phí gần như bằng không, trong khi các cách giảm thiểu tiêu tốn công sức phần mềm vô hạn.

NÓI LẠI THẬT ĐƠN GIẢN

Có một trò ảo giác thị giác quen thuộc: hình vẽ một cái hộp dây thép, và nhìn một lúc thì nó tự lật — lúc thì mặt này ở trước, lúc thì mặt kia. Bạn không điều khiển được, và không có câu trả lời đúng, vì bản thân hình vẽ chứa đúng hai cách hiểu và cả hai đều khớp hoàn hảo với những gì trên giấy. Một cái tag phẳng nhìn qua một camera rơi đúng vào tình huống ấy. Từ hình dạng méo của cái tag trên ảnh, có đúng hai cách nghiêng nó trong không gian để cho ra hình dạng đó — nghiêng về phía này, hoặc nghiêng về phía kia. Máy tính cũng không có câu trả lời đúng, và nó cũng lật qua lật lại, chỉ có điều nó lật mấy chục lần mỗi giây thay vì mấy giây một lần.

Điều tệ nhất bạn có thể làm là lấy trung bình hai cách hiểu. Trung bình của “nghiêng sang trái” và “nghiêng sang phải” là “không nghiêng gì cả” — một câu trả lời trơn tru, ổn định, và luôn luôn sai. Nhưng nó lại là điều đầu tiên ai cũng làm, vì con số ngừng nhảy và mọi thứ trông đã ổn.

Có hai cách xử lý đúng, và chúng rất khác nhau về bản chất. Cách thứ nhất là dạy máy tính nói “tôi không chắc” — đo xem hai cách hiểu khớp với ảnh tốt ngang nhau tới mức nào, và khi chúng ngang nhau thì im lặng thay vì đoán. Cách này luôn dùng được nhưng nó không giải quyết gì, nó chỉ trung thực.

Cách thứ hai là làm cho câu hỏi hết mơ hồ. Với cái hộp dây thép, chỉ cần vẽ thêm một chỗ mà một cạnh che khuất cạnh kia — chỉ một chỗ thôi — và ảo giác biến mất vĩnh viễn, vì bây giờ chỉ còn một cách hiểu khớp. Với cái tag, việc tương đương là làm cho nó dừng phẳng nữa: kẻ nghiêng vài cái tag, hoặc đặt một cái nhô ra trước những cái kia. Và đây là điều đáng nhớ nhất của cả chương: việc ấy được làm bằng một cái nẹp nhôm và bốn con vít, không phải bằng một thuật toán.

Chỗ không rút gọn được: ranh giới giữa “đủ không đồng phẳng” và “chưa đủ” thì không có công thức. Nó phụ thuộc vào nhiễu của hệ bạn, vào kích thước tag trên ảnh, vào khoảng cách — nên nó phải đo, không suy ra được. May là đo bằng mô phỏng rẻ và làm được trong một buổi.

Một câu: *Cái tag phẳng của bạn luôn có hai câu trả lời; chọn đúng là một trò may rủi, còn đặt nó nghiêng đi thì chỉ còn một câu.*

1.9 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *target phẳng có đúng hai nghiệm cục bộ, và tập điểm không đồng phẳng có nghiệm duy nhất* — là mệnh đề tôi tin nhất trong cả cây, vì nó đã qua cửa (c) với ba nguồn độc lập cách nhau ba thập niên [1]–[3], và cửa (a) bằng lập luận đối xứng. Nó bị bác bỏ nếu ai đó trình bày một cấu hình target phẳng không suy biến mà bài toán PnP chỉ có một cực tiểu, hoặc ngược lại một tập điểm không đồng phẳng đủ tổng quát mà vẫn có hai nghiệm phân biệt được với sai số tái chiếu bằng nhau.

Mệnh đề thứ hai — *trung bình hai nghiệm luôn rơi vào fronto-parallel* — bị bác bỏ nếu mô phỏng cho thấy trung bình của hai nghiệm IPPE trên một chuỗi khung hình hội tụ về một giá trị khác $\alpha = 0$ một cách có hệ thống. Đây là mệnh đề tôi dẫn từ hình học của mục 2 (cửa a) và nó *chưa* qua cửa (b) — một chỗ nọ kiểm chứng, tuy nhỏ vì nó là hệ quả trực tiếp của quan hệ gương.

Mệnh đề thứ ba — *thêm điểm đồng phẳng không cải thiện ρ* — là mệnh đề dễ kiểm nhất và cũng là mệnh đề có giá trị thực hành cao nhất, vì nếu nó sai thì lời khuyên trung tâm của chương (phá đồng phẳng thay vì thêm tag) sẽ sai theo. Nó bị bác bỏ nếu mô phỏng cho thấy ρ tăng đáng kể khi đi từ 4 lên 64 góc đồng phẳng ở cùng một góc nghiêng. Mini project ở trên chính là phép kiểm ấy, và nó nên được chạy sớm.

Cuối cùng, một mệnh đề mềm hơn: chương này khẳng định rằng *ambiguity là nguồn lỗi xoay lớn hơn nhiều trong bài toán docking điển hình*. Điều đó bị bác bỏ nếu trong một hệ thực, sau khi đã khử hoàn toàn ambiguity bằng constellation không đồng phẳng, sai số xoay vẫn không cải thiện đáng kể — khi ấy nút thắt nằm ở chỗ khác (nhiều khả năng là sai số cơ khí của constellation, B/12). (Tôi phát biểu mệnh đề này dựa trên cấu trúc của vấn đề chứ không dựa trên thống kê ca hòng thực tế; tôi không có số liệu về tỉ lệ.)

1.10 Kết nối

Chương này là chương bệnh lý của Phần A, và nó nối vào cây ở nhiều chỗ hơn độ dài của nó gợi ý.

Nối lên trên. A/02-homography-va-hinh-hoc-phang là nơi hai nghiệm xuất hiện lần đầu dưới dạng đại số: phép phân rã homography thành pose vốn đã cho nhiều nghiệm, và chương này giải thích ý nghĩa hình học của chúng. A/03-pnp-va-uoc-luong-tu-the cho IPPE và sai số tái chiếu, tức cả hai công cụ mà chương này dùng để đo ambiguity; nó cũng là nơi giải thích vì sao phải khởi tạo bằng lời giải dạng đóng thay vì tùy tiện.

Nối ngang, và đây là liên kết quan trọng nhất. A/05-lan-truyen-sai-so-pixel-sang-pose là chương song sinh: cùng một sự kiện hình học — độ nhạy tỉ lệ với $\sin \alpha$ — được nhìn từ hai phía. Chương 5 nói “ma trận thông tin gần suy biến theo hướng nghiêng, nên hiệp phương sai lớn”; chương này nói “có hai cực tiểu gần nhau, nên bộ giải nhảy giữa chúng”. Cả hai đều đúng và chúng mô tả cùng một bệnh. Một phân tích chỉ dùng chương 5 sẽ kết luận rằng sai số xoay là nhiều và có thể lấy trung bình — một kết luận sai và tốn kém. Đó là lý do hai chương phải đọc cùng nhau.

Nối xuống dưới. B/09-multi-marker-constellation là chương biến mục 6 thành thiết kế cụ thể: nghiêng bao nhiêu độ, đặt tag ở đâu, độ lệch khỏi mặt phẳng cần bao nhiêu. B/10-xu-ly-ambiguity nhận toàn bộ mục 5 và mục 8: nó là chương về những gì phải làm ở tầng code khi bạn *chưa* khử được ambiguity bằng hình học — ngưỡng, từ chối cập nhật, nhất quán thời gian, fuse IMU. B/11-uoc-luong-theo-thoi-gian nhận cảnh báo ở mục 5: mọi kỹ thuật làm mượt trong chương ấy chỉ hợp lệ *sau khi* nghiệm đã được chọn tin cậy, và chương này là lý do của điều kiện ấy.

Cuối cùng, một nối sang Phần C. C/15-ngan-sach-sai-so phải xử lý ambiguity theo một cách khác với mọi dòng khác trong ngân sách: nó không phải một nguồn sai số cộng vào mà là một *lỗi thô* phá hỏng toàn bộ ngân sách khi nó xảy ra. Ngân sách sai số giả định bạn đang ở gần nghiệm đúng; nếu bạn đang ở nghiệm sai thì mọi con số milimét trong bảng đều vô nghĩa. Đó là lý do việc khử ambiguity phải là điều kiện tiên quyết chứ không phải một dòng trong bảng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao thêm mười hai góc đồng phẳng không cải thiện tỉ số ρ trong khi nó rõ ràng cải thiện σ — hãy trả lời bằng cách chỉ ra phép biến đổi nào đang được bảo toàn?
2. Một đồng nghiệp thấy pose nhảy và đề xuất dùng bộ lọc Kalman. Nêu chính xác chuyện gì sẽ xảy ra với đầu ra, và vì sao kết quả trông tốt hơn nhưng thực chất tệ hơn.
3. Hai nghiệm cách nhau 2α . Vì sao biên độ nhảy *nhỏ* lại là dấu hiệu xấu chứ không phải dấu hiệu tốt?
4. Bạn dùng ống kính tele để có f lớn, theo lời khuyên của chương 5. Điều đó ảnh hưởng thế nào tới ambiguity, và bạn cân bằng hai yêu cầu ngược nhau ấy bằng cách nào?
5. Bạn đã phá tính đồng phẳng bằng cách đặt một tag nhô ra trước 3 mm so với các tag khác. Vì sao điều đó có thể vẫn không đủ, và bạn kiểm bằng cách nào mà không cần dựng phần cứng?
6. Tiêu chí “chọn nghiệm gần pose khung trước” có một chế độ hỏng rất đặc trưng. Mô tả nó, và nêu một cơ chế phát hiện được nó từ trong dữ liệu đo.
7. Vì sao ambiguity không thể được coi là một dòng trong bảng ngân sách sai số, mà phải được coi là điều kiện tiên quyết?

Nguồn

- [1] T. Collins and A. Bartoli, “Infinitesimal Plane-Based Pose Estimation,” *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014, mức tin A; IPPE — hai nghiệm cho target phẳng, là nền của SOLVEPNP_IPPE trong OpenCV.
- [2] D. Oberkamp, D. F. DeMenthon, and L. S. Davis, “Iterative Pose Estimation Using Coplanar Feature Points,” *Computer Vision and Image Understanding*, vol. 63, no. 3, pp. 495–511, 1996, mức tin A; hai nghiệm cho điểm đồng phẳng, trước Schweighofer.
- [3] G. Schweighofer and A. Pinz, “Robust Pose Estimation from a Planar Target,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 12, pp. 2024–2030, 2006, mức tin A; phân tích hai cực tiểu của hàm mục tiêu với target phẳng — nguồn gốc của pose ambiguity.
- [4] B. Benligiray, C. Topal, and C. Akinlar, “STag: A Stable Fiducial Marker System,” in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.
- [5] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền án nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.